Distributed data management

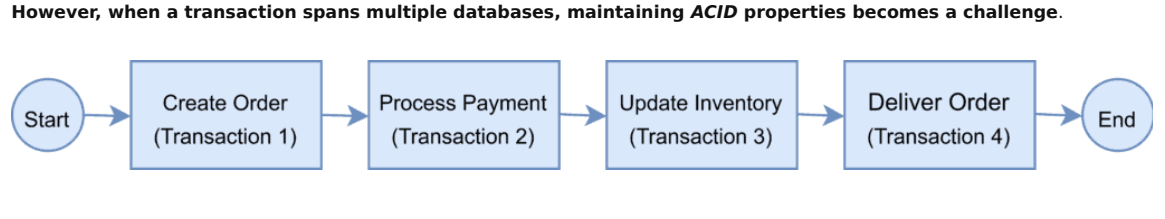
While distributed applications provides numerous benefits, they have several limitations as well. One of the major problems is how to handle a transaction that spans multiple services.

Database per Service Pattern

The *Database per Service* pattern is a key principle in microservices architecture that promotes loose coupling and independent scalability by assigning each microservice its own dedicated database.

- Services can use the most suitable database technology (SQL, NoSQL, time-series, etc.) for their specific needs.
- Services can scale their data stores independently and remain insulated from failures in other services.

However, when a transaction spans multiple databases, maintaining *ACID* properties becomes a challenge.


Start
Create Order (Transaction 1)
Process Payment (Transaction 2)
Update Inventory (Transaction 3)
Deliver Order (Transaction 4)
End

The **SAGA pattern** replaces a single distributed transaction with a sequence of local transactions across services. Each step is independent, and if something fails, **compensating actions** are executed to undo previous steps and keep the system consistent.

The **CQRS pattern** improves handling of complex systems by separating **writes (commands)** from **reads (queries)**. This allows each side to be optimized, scaled, and implemented independently, improving both **scalability and performance**.

CAP Theorem

In 2000, Eric Brewer introduced the idea that there is a fundamental trade-off between consistency, availability, and partition tolerance in distributed systems. More specifically, the CAP theorem states that **any distributed system can simultaneously provide only two of three guarantees**:

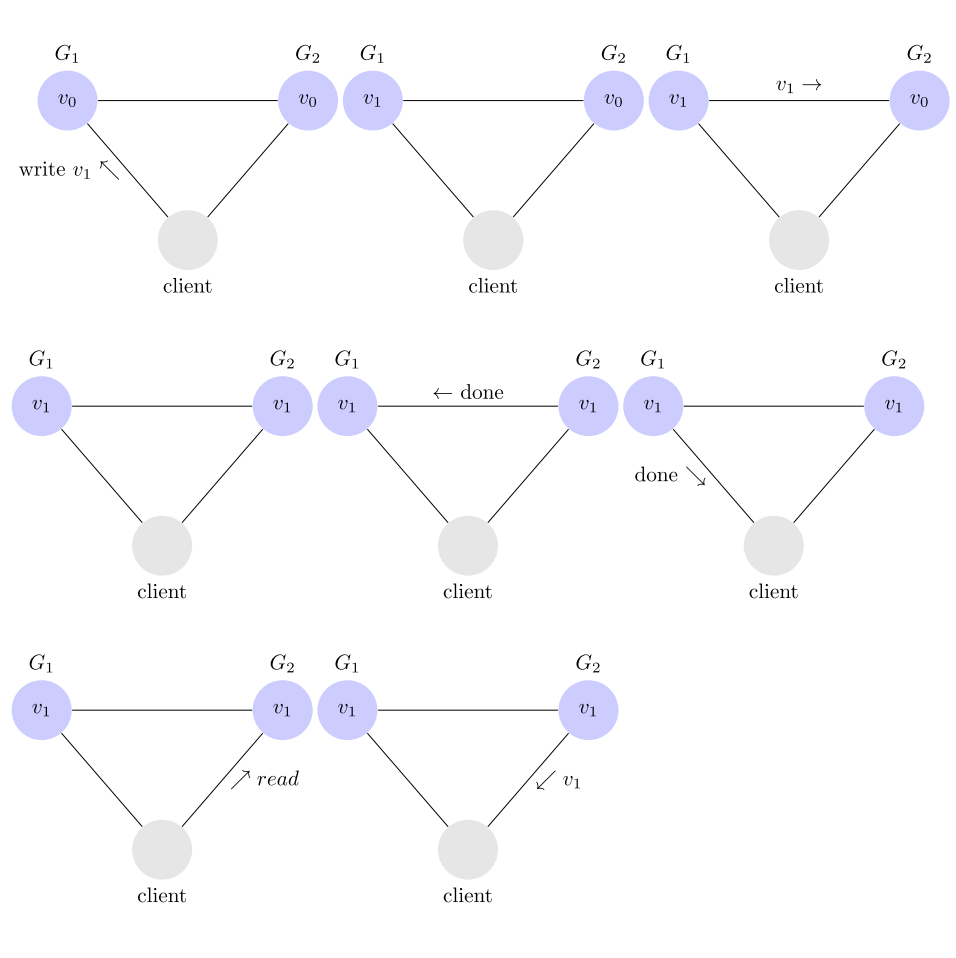
- Consistency
- Availability
- Partition tolerance

Consistency

> *Every read receives the most recent write or an error.*

In a consistent system, once a client writes a value to any server and gets a response, it expects to get that value (or a fresher value) back from any server it reads from.

Here is an example of a **consistent** system.


G_1
G_2
v_0
v_1
write v_1
$v_1 \rightarrow$
$\leftarrow$ done
done
read
client

In this system, G1 replicates its value to G2 before sending an acknowledgement to the client. Thus, when the client reads from G2, it gets the most recent value of v: v1.

Availability

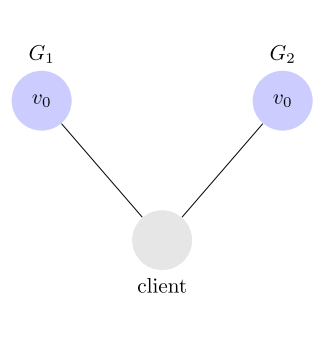
> *Every request received by a non-failing node in the system must result in a response.*

In an available system, if our client sends a request to a server and the server has not crashed, then the server must eventually respond to the client. The server is not allowed to ignore the client's requests.

Partition Tolerance

> *The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes.*

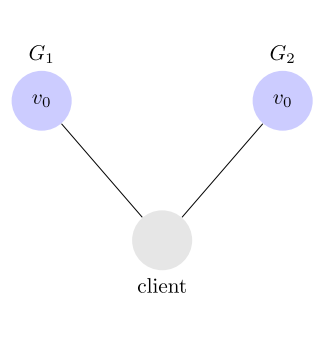
This means that any messages G1 and G2 send to one another can be dropped. If all the messages were being dropped, then our system would look like this.


G_1
G_2
v_0
client

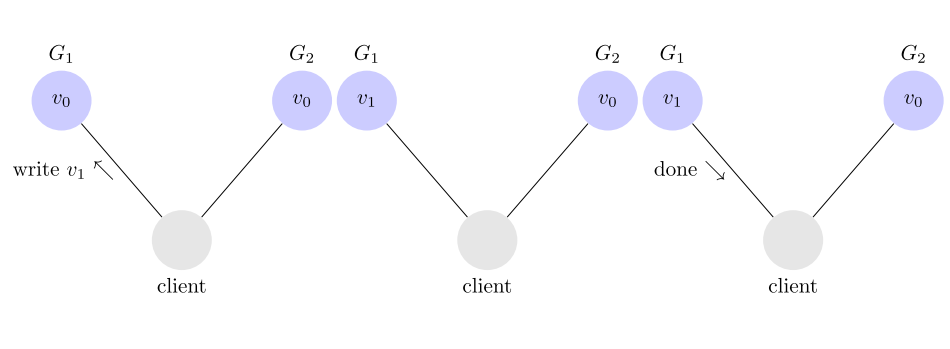
Our system has to be able to function correctly despite arbitrary network partitions in order to be partition tolerant.

The Proof

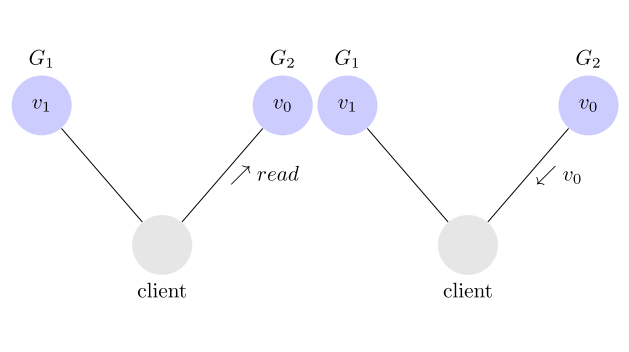
Assume for contradiction that there does exist a system that is consistent, available, and partition tolerant. The first thing we do is partition our system. It looks like this.


G_1
G_2
v_0
client

Next, we have our client request that v1 be written to G1. Since our system is available, G1 must respond. Since the network is partitioned, however, G1 cannot replicate its data to G2.


G_1
G_2
v_0
v_1
write v_1
done
client

Next, we have our client issue a read request to G2. Again, since our system is available, G2 must respond. It returns v0. **This is inconsistent.**

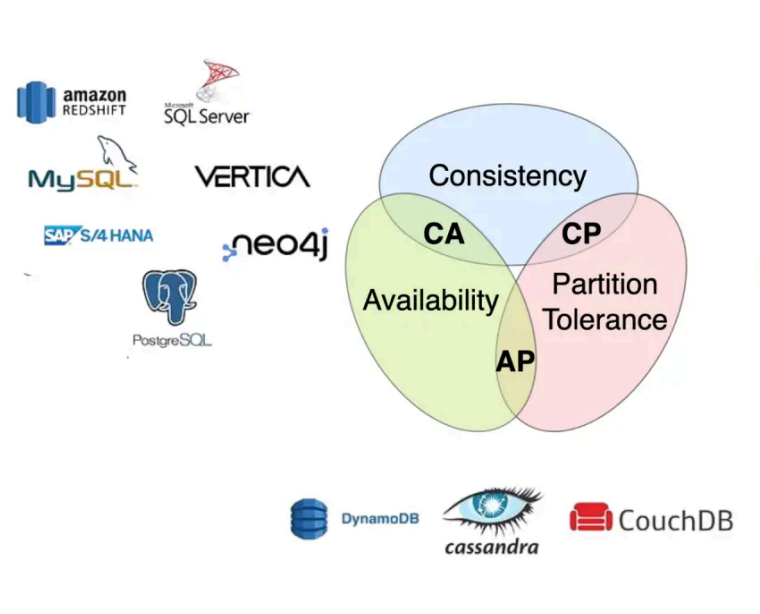

G_1
G_2
v_1
v_0
read
client

We assumed a consistent, available, partition tolerant system existed, but we just showed that there exists an execution for any such system in which the system acts inconsistently. **Thus, no such system exists.**

Key Insight

In a distributed system, network partitions are inevitable. Under normal conditions (i.e., no network failures), systems strive to provide both **Consistency** and **Availability**. However, when a partition occurs, a trade-off must be made:

1. **AP systems** prioritize **availability** by continuing to accept reads and writes during a partition, potentially at the cost of **consistency** (leading to **eventual consistency**).
2. **CP systems** prioritize **consistency** by rejecting some operations—typically writes—during a partition, ensuring that any served data remains consistent.
3. **CA systems** maintain both **consistency** and **availability** in the absence of network partitions. When a partition occurs, they must choose to behave as either **AP** or **CP**.


Consistency
CA
CP
Availability
Partition Tolerance
AP
amazon REDSHIFT
SQL Server
MySQL
VERTICA
SAP S/4 HANA
neo4j
PostgreSQL
redis
mongoDB
APACHE HBASE
DynamoDB
cassandra
CouchDB

CA Systems (Consistency and Availability)

- **Definition**: CA systems ensure **Consistency** and **Availability** but do not provide **Partition Tolerance**. CA systems are ideal for environments where network reliability is high, and both consistency and availability are critical.
- **Examples**:
  - **Centralized, tightly coupled distributed systems** (e.g., MySQL, PostgreSQL) within stable network environments.

CP Systems (Consistency and Partition Tolerance)

- **Definition**: CP systems ensure **Consistency** and **Partition Tolerance** but do not guarantee **Availability** during network partitions. CP systems are suitable for applications where data accuracy and consistency are critical, even at the cost of availability during network issues.
- **Examples**:
  - **Zookeeper**, **Redis** (in specific configurations): Systems that prioritize strong consistency and may block requests if partitioned to ensure all nodes stay synchronized.

AP Systems (Availability and Partition Tolerance)

- **Definition**: AP systems ensure **Availability** and **Partition Tolerance** but do not guarantee **Consistency**. AP systems are suitable for applications that require high availability and can tolerate temporary inconsistencies.
- **Examples**:
  - **Cassandra**, **DynamoDB**: NoSQL databases that prioritize availability and partition tolerance, offering eventual consistency.

PACELC Theorem

In 2007, **Daniel Abadi** extended the CAP theorem to describe a more complete trade-off in distributed systems.

- **PACELC** states that in a distributed system:
- **If there is a Partition (P), choose between Availability (A) and Consistency (C)**.
- **Else (E), choose between Latency (L) and Consistency (C).**

Partition Scenario (P)

- Like CAP, when a network partition occurs:
  - Either prioritize **consistency** (block some operations) → CP behavior
  - Or prioritize **availability** (accept all requests) → AP behavior

Else Scenario (E)

- When there is **no partition**, systems still face a trade-off:
  - **Consistency (C)**: ensure all replicas are fully synchronized before responding → may increase **latency**
  - **Latency (L)**: respond quickly, possibly before all replicas are updated → leads to **eventual consistency**
- This trade-off occurs **even in healthy networks**, because data replication across nodes is never instantaneous.

Examples

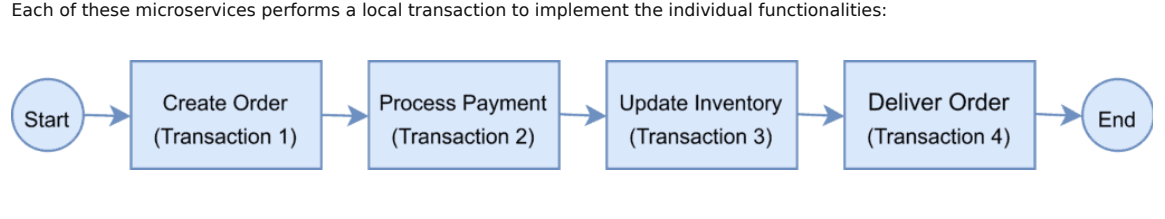
Database / System	Partition (P)	Else / No Partition (E)	Notes
Cassandra	A	L	Prioritizes **Availability** during partitions; prioritizes **low latency** and eventual consistency when network is healthy.
DynamoDB	A	L	Similar to Cassandra: highly available and fast, with eventual consistency when no partitions.
HBase	C	C	Prioritizes **Consistency** both during partitions and when network is healthy; may sacrifice availability or latency.
MongoDB	C	C	Strong consistency focus, both during partitions and in normal operation (configurable with replica set settings).
Zookeeper	C	C	Strong consistency system, may block writes during partitions; consistency prioritized even when network is healthy.
Redis (config-dependent)	C	C / L	Can be configured for strong consistency (CP) or faster, eventually consistent responses.

Challenges of Distributed Transaction

We'll take an example of an e-commerce application that processes online orders and is implemented as a microservice architecture.

There is a microservice to create the orders, one that processes the payment, another that updates the inventory and the last one that delivers the order.

Each of these microservices performs a local transaction to implement the individual functionalities:


Start
Create Order (Transaction 1)
Process Payment (Transaction 2)
Update Inventory (Transaction 3)
Deliver Order (Transaction 4)
End

This is an example of a distributed transaction as the transaction boundary crosses multiple services and databases.

Maintaining ACID Properties

Ensuring transaction correctness in a distributed system requires adherence to ACID principles:

- **Atomicity**: A transaction must be all-or-nothing—either all steps complete successfully, or none take effect.
- **Consistency**: The system must transition from one valid state to another, preserving data integrity.
- **Isolation**: Transaction isolation defines how visible data modifications are when multiple services access the same data concurrently. For example, if a microservice updates a record while another service reads it, should the second service see the old or the new value?
- **Durability**: Once a transaction is committed, its changes must persist, even in the event of a system failure.

To ensure a successful order processing:

- all four microservices must complete the individual local transactions
- if any of the microservices fail to complete its local transaction, all the completed preceding transactions should roll back to ensure data integrity.

The Two-Phase Commit Pattern (2PC)

The **Two-Phase Commit (2PC)** is a widely used protocol for implementing **distributed transactions**, especially in microservice architectures.

It ensures that a transaction involving multiple services either **commits everywhere or aborts everywhere**.

Main Components

- **Coordinator**
  - Orchestrates the entire transaction
  - Decides whether to commit or abort
- **Participants**
  - Microservices involved in the transaction
  - Execute local parts of the transaction

The Two Phases

1. Phase 1 — Prepare (Voting Phase)

The coordinator asks all participants:

> *"Can you commit the transaction?"*

Each participant replies:

- **YES** → ready to commit
- **NO** → cannot commit

---

2. Phase 2 — Commit / Abort

- If **all participants respond YES**, the coordinator sends a **commit** request
- If **at least one participant responds NO**, the coordinator sends an **abort/rollback** request to all

---

Problems with 2PC

Despite its simplicity, 2PC has several important drawbacks:

- **Single Point of Failure**
  - The coordinator is critical; if it fails, the process may stall
- **Blocking Problem**
  - After voting YES, participants enter an uncertain state
  - They cannot decide independently
  - If the coordinator crashes, they may remain blocked indefinitely
- **Poor Performance and Scalability**
  - The overall transaction is limited by the slowest participant
- **Limited Database Support**
  - Many NoSQL systems do not support distributed transactions

---

The Three-Phase Commit Pattern (3PC)

The **Three-Phase Commit (3PC)** improves 2PC by adding an extra phase to reduce blocking and improve fault tolerance.

The key idea is:

> *Participants should be able to reach a safe decision even if the coordinator fails.*

The Three Phases

1. CanCommit (Voting Phase)

The coordinator asks participants whether they can commit.

Participants respond:

- **YES / NO**

---

2. PreCommit Phase

If all participants vote YES, the coordinator sends a **PRE-COMMIT** message.

At this stage:

- resources are locked
- participants prepare to commit
- but the transaction is not yet finalized

Participants acknowledge this state.

---

3. DoCommit (Final Commit Phase)

The coordinator sends the final **COMMIT** command.

If failures occur before this phase, participants can safely abort.

Advantages of 3PC over 2PC

- Reduces the **blocking problem**
- Participants can infer safe states in some failure scenarios
- Better fault tolerance than 2PC

Limitations of 3PC

Despite improvements, 3PC still has drawbacks:

- Assumes a **partially synchronous system** (bounded network delays), which is unrealistic in many real-world systems
- Introduces **additional communication overhead**
- The coordinator is still a potential **single point of failure**
- Rarely used in practice in modern distributed systems

2PC / 3PC and the CAP Theorem

Both protocols fall under the **CP (Consistency + Partition Tolerance)** category of the CAP theorem.

- They prioritize **consistency**
- They sacrifice **availability** during network partitions

If the coordinator or a participant becomes unreachable:

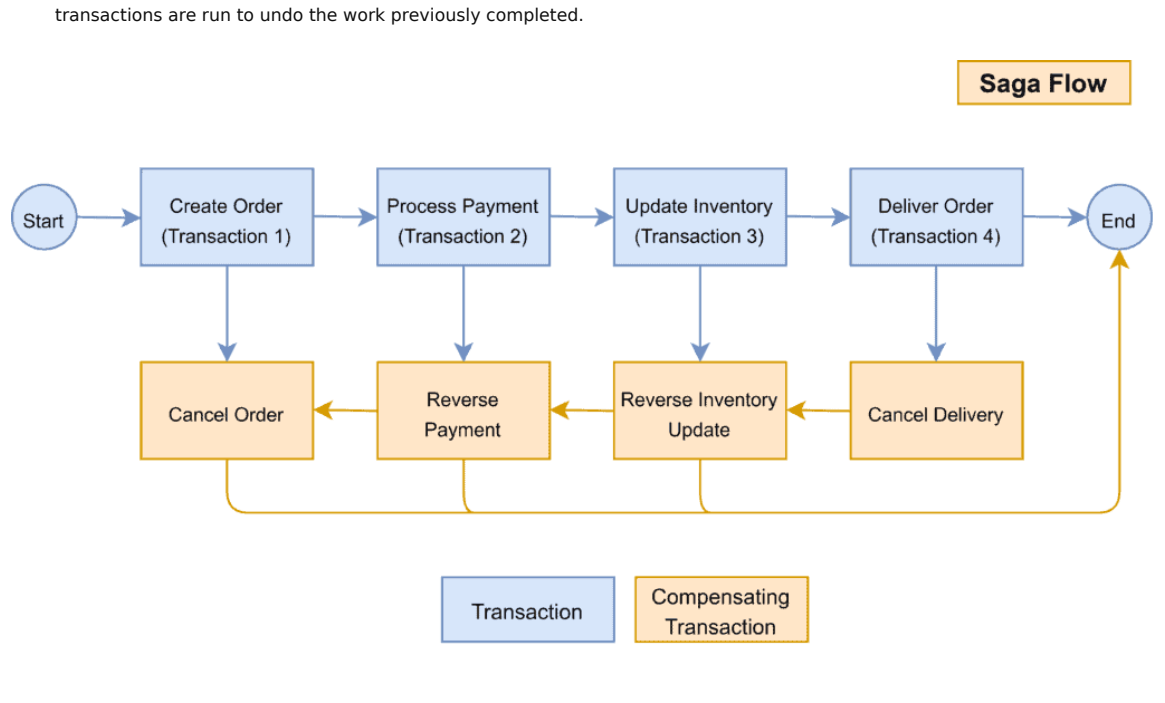
- the transaction may block
- the system cannot safely proceed

👉 Result: strong consistency, but reduced availability under failures

The SAGA Pattern

The Saga pattern, introduced in 1987 by Hector Garcia Molina & Kenneth Salem, defines a saga as a sequence of transactions that can be interleaved with one another.

- A local transaction is the unit of work performed by a Saga participant.
- Every operation that is part of the Saga can be rolled back by a compensating transaction that must be *idempotent* (*retryable*).
- The Saga pattern guarantees that either all operations complete successfully or the corresponding compensation transactions are run to undo the work previously completed.


Saga Flow
Start
Create Order (Transaction 1)
Process Payment (Transaction 2)
Update Inventory (Transaction 3)
Deliver Order (Transaction 4)
End
Cancel Order
Reverse Payment
Reverse Inventory Update
Cancel Delivery
Transaction
Compensating Transaction

Saga Pattern and CAP Theorem

The Saga pattern aligns with **AP** in the CAP theorem, favoring Availability while accepting **eventual consistency**. The Saga pattern offers **eventual consistency** through compensating transactions, ensuring the system eventually reaches a valid state without requiring strong consistency.

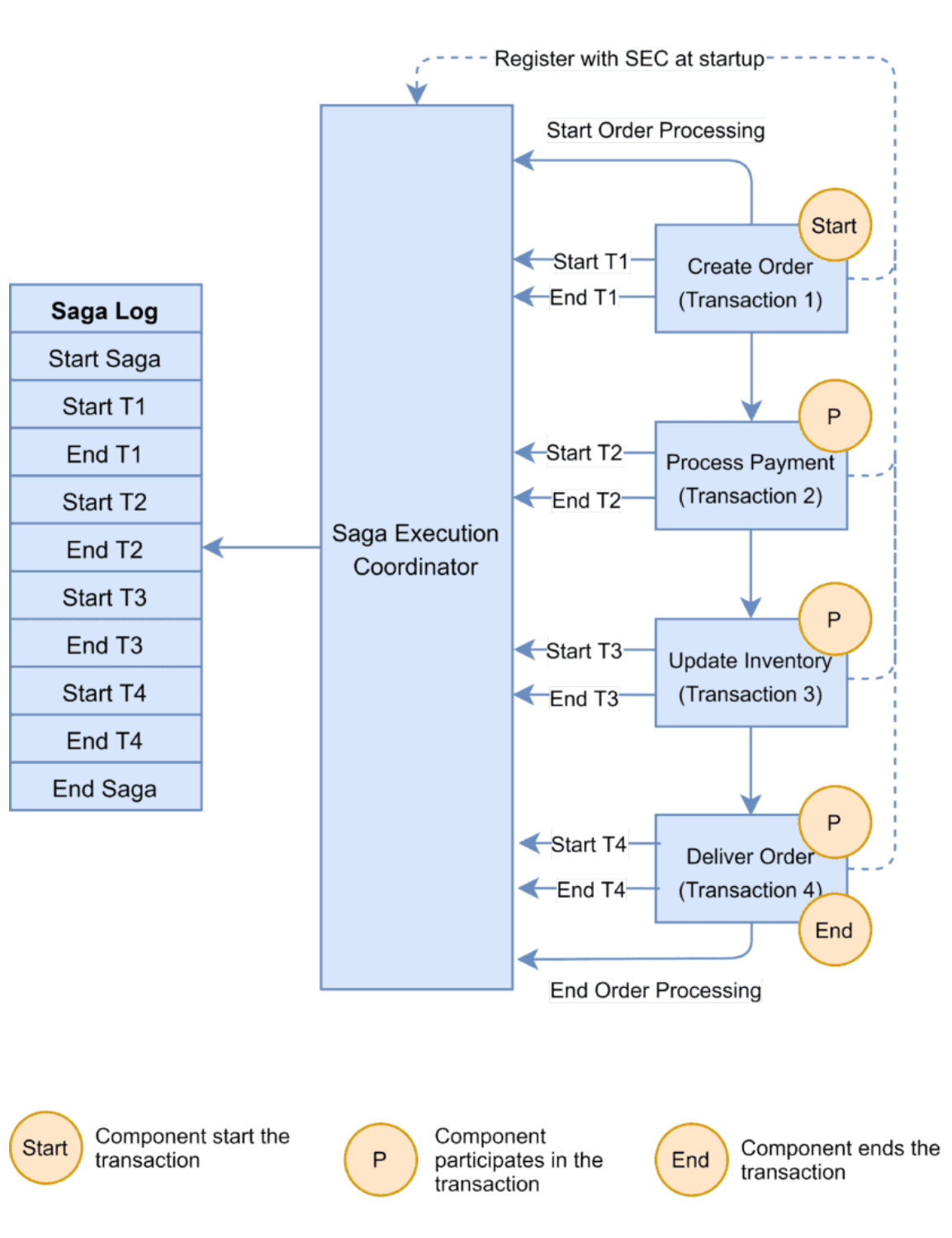
It enhances **availability** by being non-blocking—allowing operations to proceed even during failures—and supports **partition tolerance** by executing steps asynchronously without the need for global coordination, making it more resilient in distributed environments compared to protocols like 2PC.

It is ideal for systems where high availability and partition tolerance are priorities, and where eventual consistency is acceptable, such as e-commerce order processing, travel bookings, and other long-running workflows.

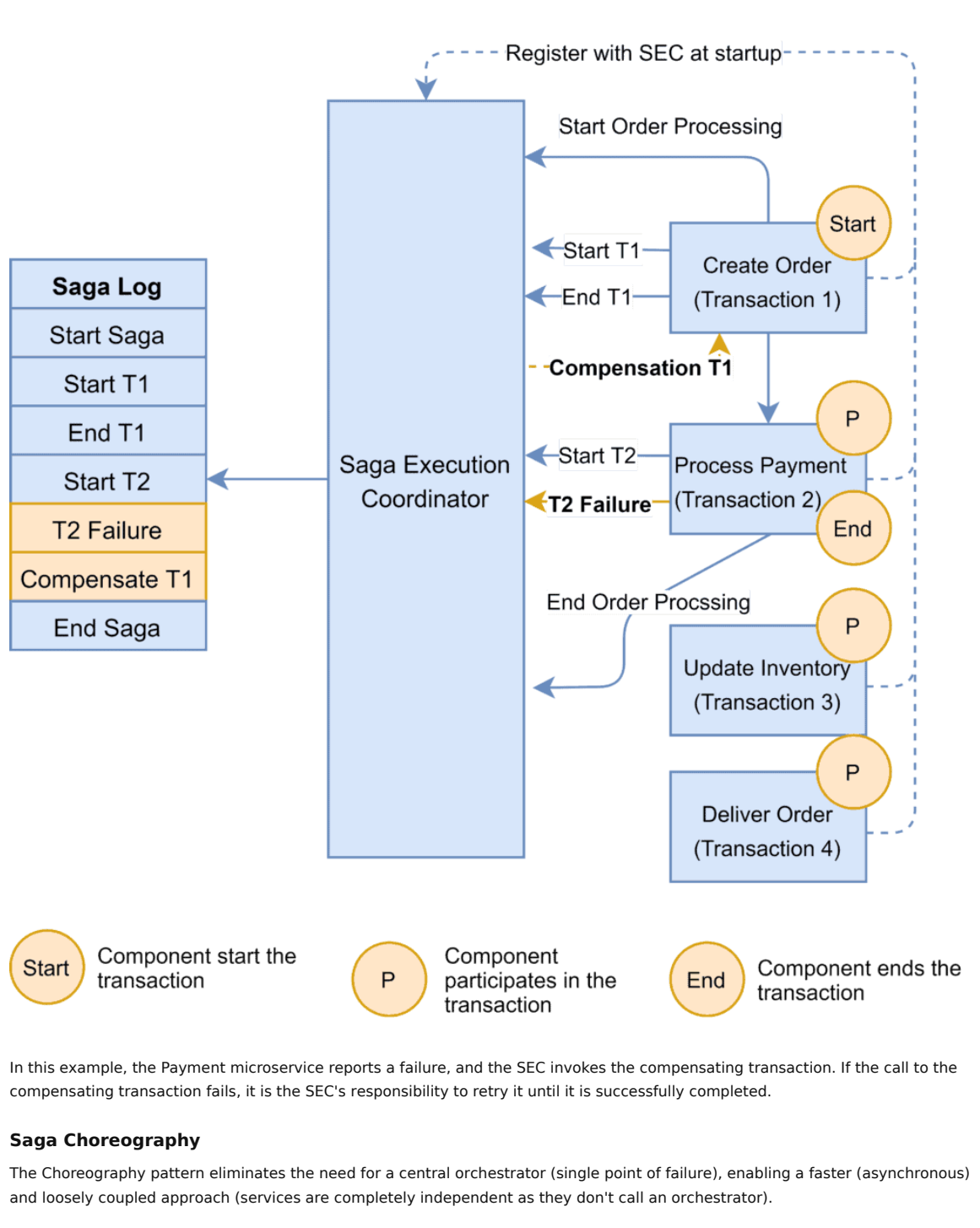
Saga Orchestration

In the Saga Orchestration pattern, the Saga Execution Coordinator (SEC) is the central component to implement a Saga flow. It contains a Saga log that captures the sequence of events of a distributed transaction.

- For any failure, the SEC inspects the Saga log to identify the impacted components and the sequence in which the compensating transactions should run.
- For any failure in the SEC, it can read the Saga log once it's coming back up. It can then identify the transactions successfully rolled back, which ones are pending, and can take appropriate actions.


Register with SEC at startup
Start Order Processing
Saga Log
Start Saga
Start T1
End T1
Start T2
End T2
Start T3
End T3
Start T4
End T4
End Saga
Saga Execution Coordinator
Start T1
End T1
Create Order (Transaction 1)
Start
Start T2
End T2
Process Payment (Transaction 2)
P
Start T3
End T3
Update Inventory (Transaction 3)
Start T4
End T4
Deliver Order (Transaction 4)
End
End Order Processing
Start
Component start the transaction
P
Component participates in the transaction
End
Component ends the transaction

In the event of a failure, the microservice reports the failure to SEC, and it is the SEC's responsibility to invoke the relevant compensation transactions:

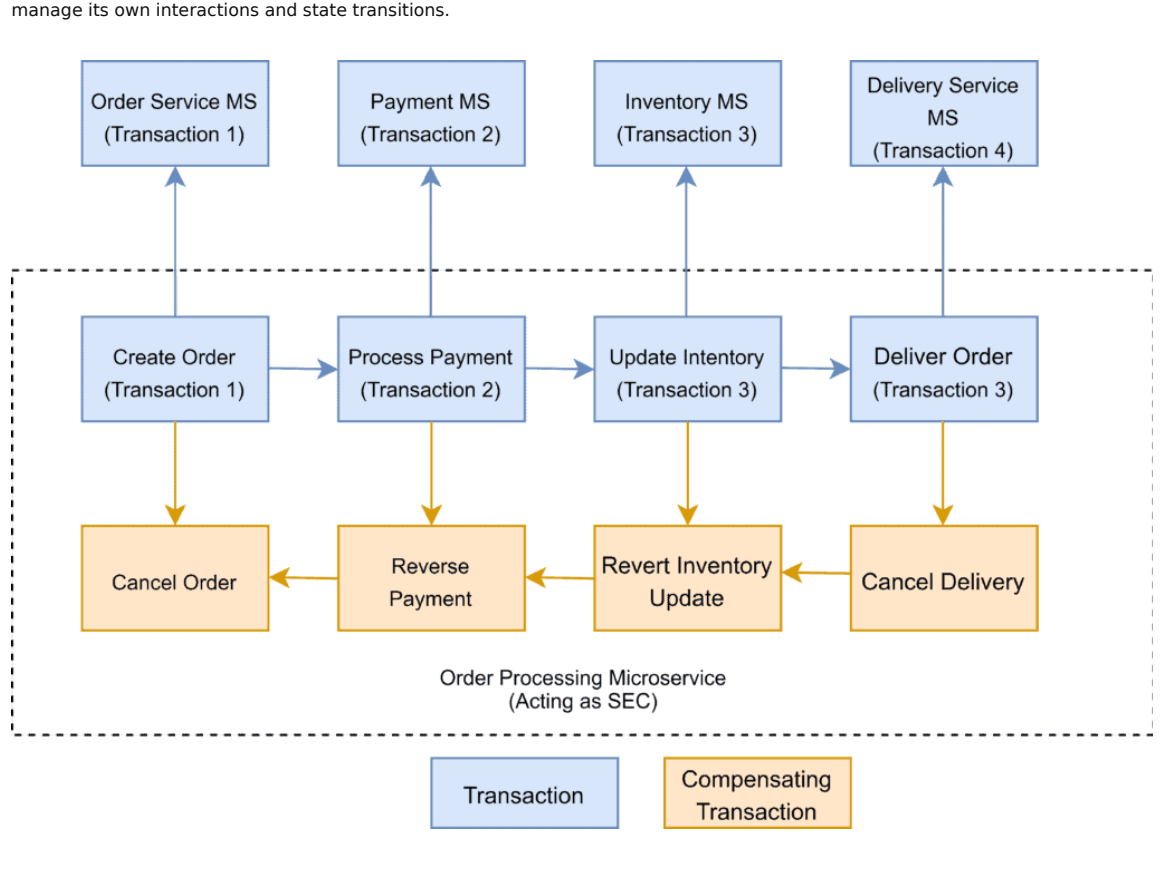

Register with SEC at startup
Start Order Processing
Saga Log
Start Saga
Start T1
End T1
Start T2
T2 Failure
Compensate T1
End Saga
Saga Execution Coordinator
Start T1
End T1
Create Order (Transaction 1)
Start
Compensation T1
Start T2
T2 Failure
Process Payment (Transaction 2)
P
End
End Order Processing
Update Inventory (Transaction 3)
Deliver Order (Transaction 4)
Start
Component start the transaction
P
Component participates in the transaction
End
Component ends the transaction

In this example, the Payment microservice reports a failure, and the SEC invokes the compensating transaction. If the call to the compensating transaction fails, it is the SEC's responsibility to retry it until it is successfully completed.

Saga Choreography

The Choreography pattern eliminates the need for a central orchestrator (single point of failure), enabling a faster (asynchronous) and loosely coupled approach (services are completely independent as they don't call an orchestrator).

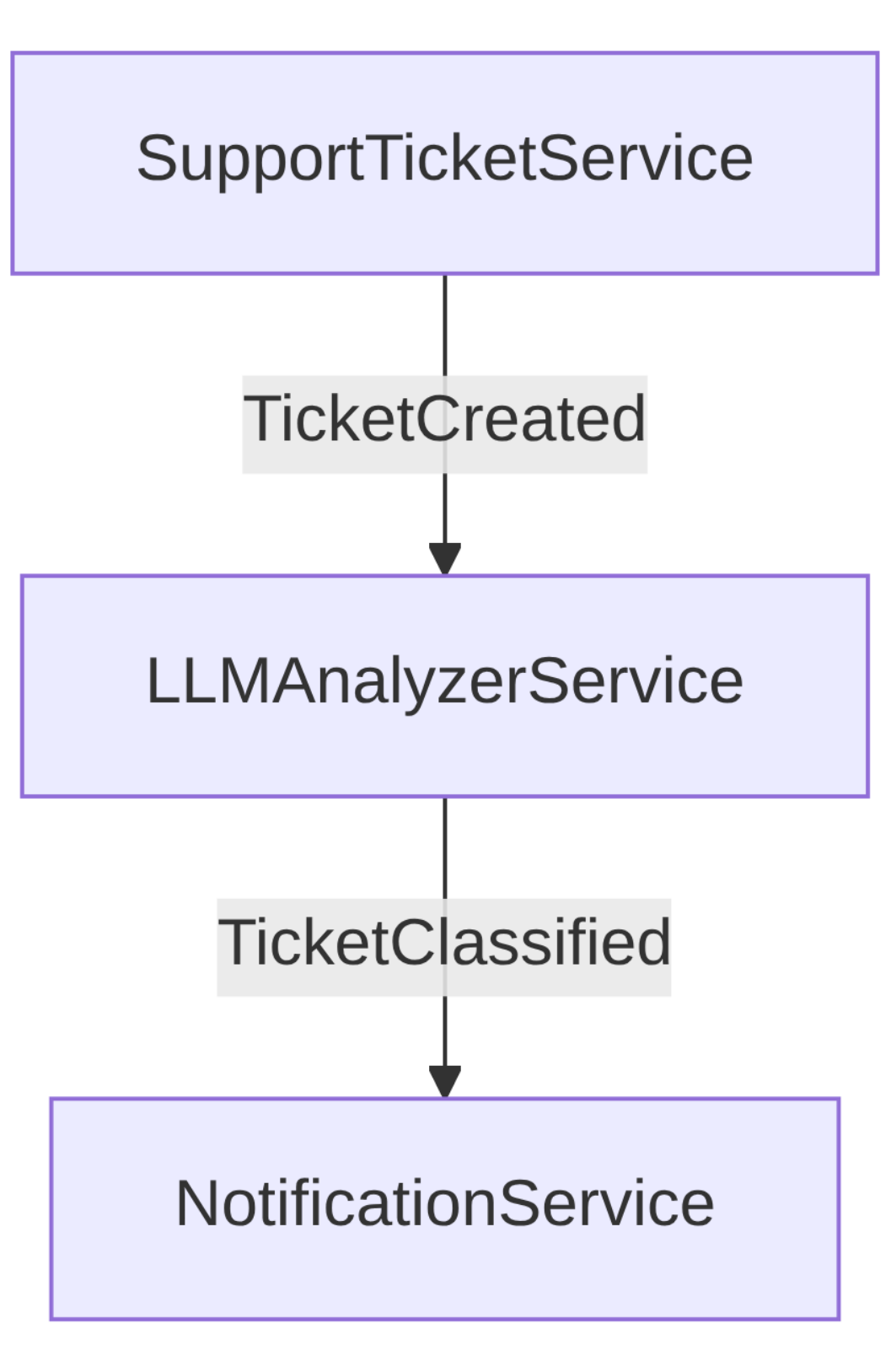
However, as the number of participants increases, the implementation becomes more **complex**, as each participant must manage its own interactions and state transitions.


Order Service MS (Transaction 1)
Payment MS (Transaction 2)
Inventory MS (Transaction 3)
Delivery Service MS (Transaction 4)
Create Order (Transaction 1)
Process Payment (Transaction 2)
Update Intentory (Transaction 3)
Deliver Order (Transaction 3)
Cancel Order
Reverse Payment
Revert Inventory Update
Cancel Delivery
Order Processing Microservice (Acting as SEC)
Transaction
Compensating Transaction

Choreography-based coordination

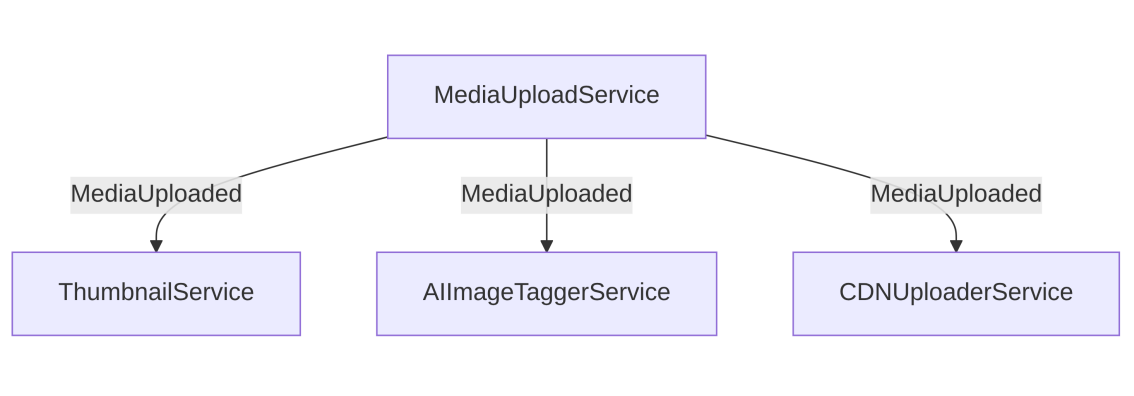
Example 1: Customer Support Triage with LLM and Notifications

- A **SupportTicketService** receives a ticket and emits `TicketCreated`.
- An **LLMAnalyzerService** listens and classifies the issue (e.g., billing, technical, emo). `TicketClassified`.
- A **NotificationService** listens to send an email/slack alert to the appropriate team.


SupportTicketService
TicketCreated
LLMAnalyzerService
TicketClassified
NotificationService

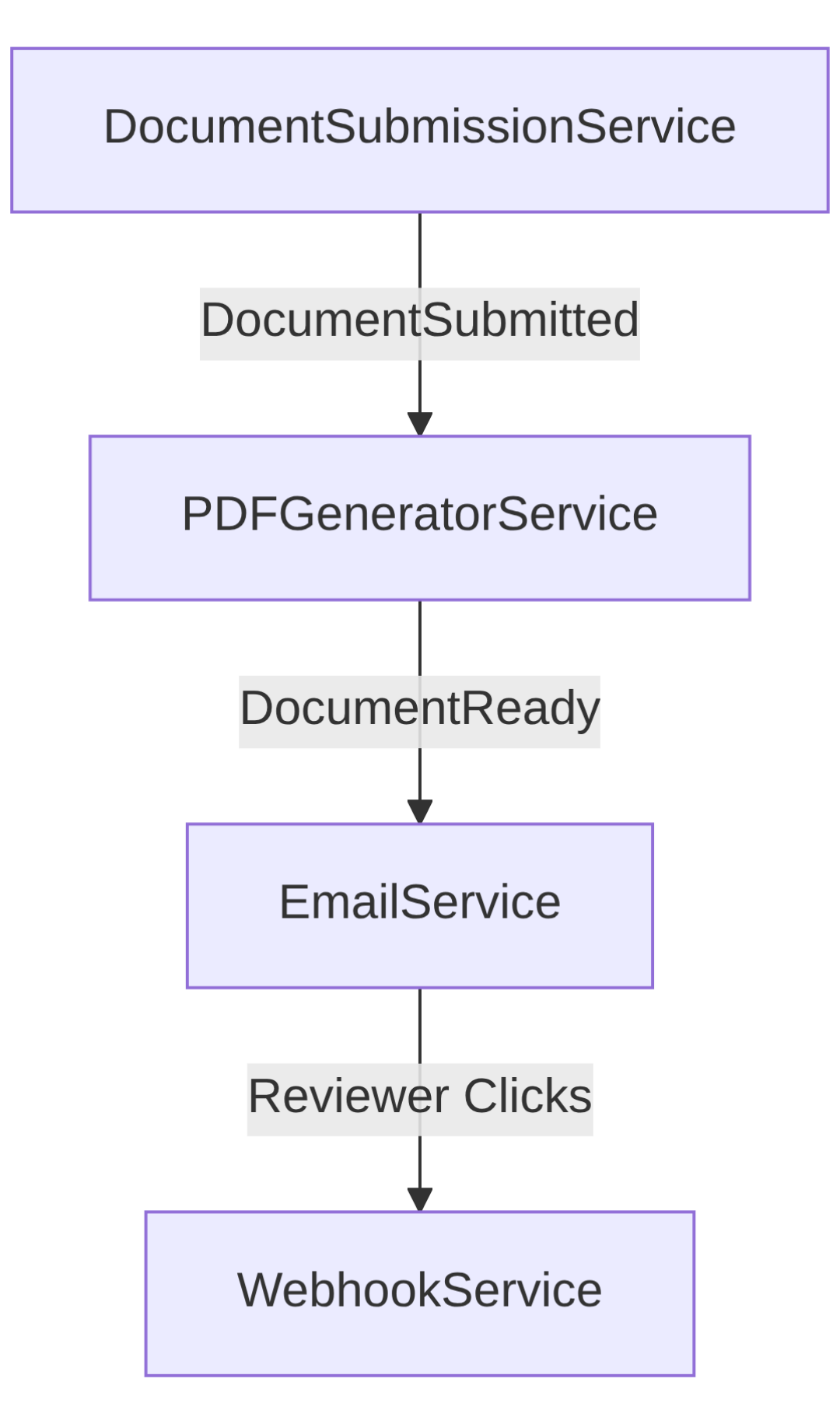
Example 2: Media Upload and Processing

- A **MediaUploadService** emits `MediaUploaded`.
- **ThumbnailService**, **AIImageTaggerService**, and **CDNUploaderService** all react in parallel:
  - generate preview
  - auto-tag images with AI
  - push to CDN


MediaUploadService
MediaUploaded
MediaUploaded
MediaUploaded
ThumbnailService
AIImageTaggerService
CDNUploaderService

Example 3: Document Approval via Email Workflow

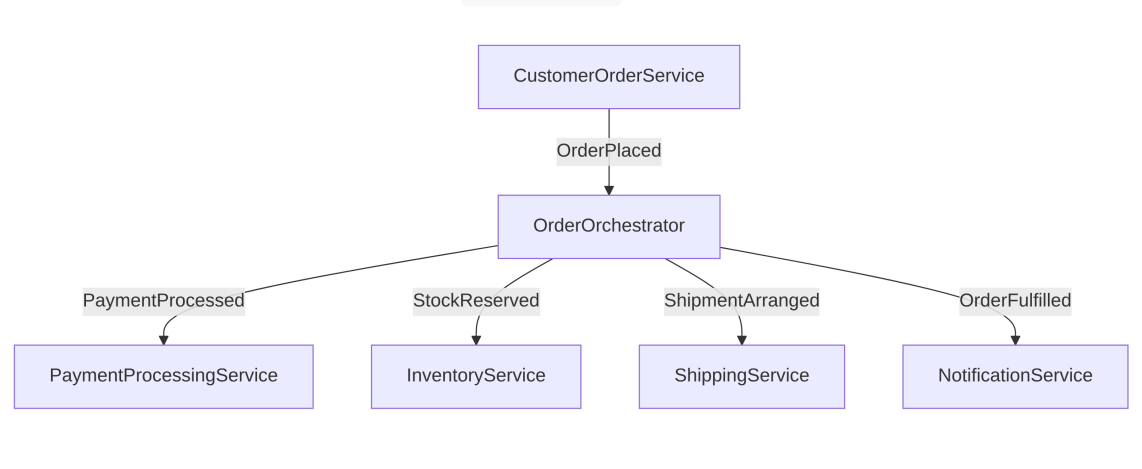
- A **DocumentSubmissionService** emits `DocumentSubmitted`.
- A **PDFGeneratorService** creates a PDF version and emits `DocumentReady`.
- An **EmailService** sends the document to a reviewer.
- A **WebhookService** receives the approval or rejection response via email click.


DocumentSubmissionService
DocumentSubmitted
PDFGeneratorService
DocumentReady
EmailService
Reviewer Clicks
WebhookService

Orchestration-based coordination

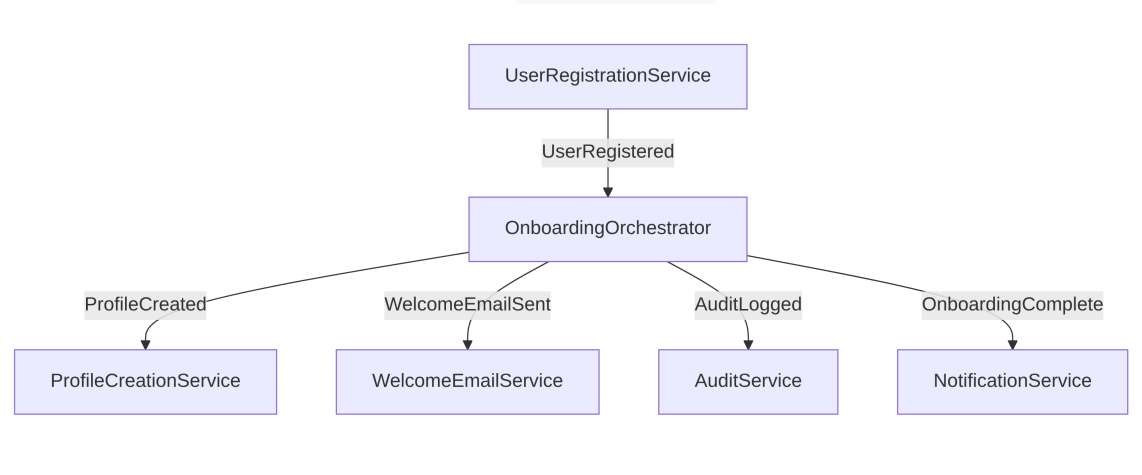
Example 1: Order Fulfillment with an Orchestrator

- A **CustomerOrderService** emits `OrderPlaced`.
- An **OrderOrchestrator** listens to this event, then triggers:
  - **PaymentProcessingService** to process the payment.
  - **InventoryService** to reserve items.
  - **ShippingService** to arrange delivery.
- Once all services finish, the orchestrator emits `OrderFulfilled`.


CustomerOrderService
OrderPlaced
OrderOrchestrator
PaymentProcessed
StockReserved
ShipmentArranged
OrderFulfilled
PaymentProcessingService
InventoryService
ShippingService
NotificationService

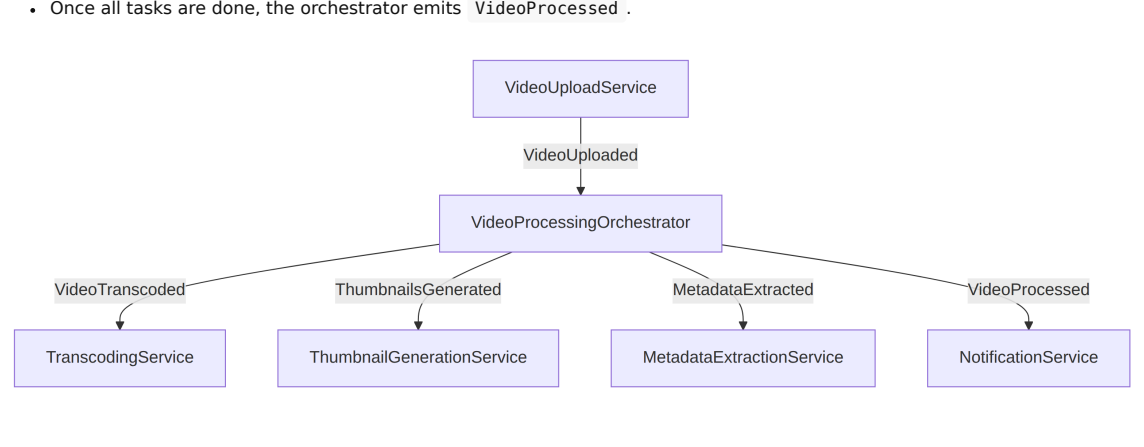
Example 2: User Onboarding Process with an Orchestrator

- A **UserRegistrationService** emits `UserRegistered`.
- An **OnboardingOrchestrator** listens and triggers:
  - **ProfileCreationService** to create a user profile.
  - **WelcomeEmailService** to send a welcome email.
  - **AuditService** to log the process.
- Once all tasks are completed, the orchestrator emits `OnboardingComplete`.


UserRegistrationService
UserRegistered
OnboardingOrchestrator
ProfileCreated
WelcomeEmailSent
AuditLogged
OnboardingComplete
ProfileCreationService
WelcomeEmailService
AuditService
NotificationService

Example 3: Video Processing with an Orchestrator

- A **VideoUploadService** emits `VideoUploaded`.
- A **VideoProcessingOrchestrator** listens to this event and triggers:
  - **TranscodingService** to transcode the video.
  - **ThumbnailGenerationService** to create thumbnails.
  - **MetadataExtractionService** to analyze video metadata.
- Once all tasks are done, the orchestrator emits `VideoProcessed`.


VideoUploadService
VideoUploaded
VideoProcessingOrchestrator
VideoTranscoded
ThumbnailsGenerated
MetadataExtracted
VideoProcessed
TranscodingService
ThumbnailGenerationService
MetadataExtractionService
NotificationService

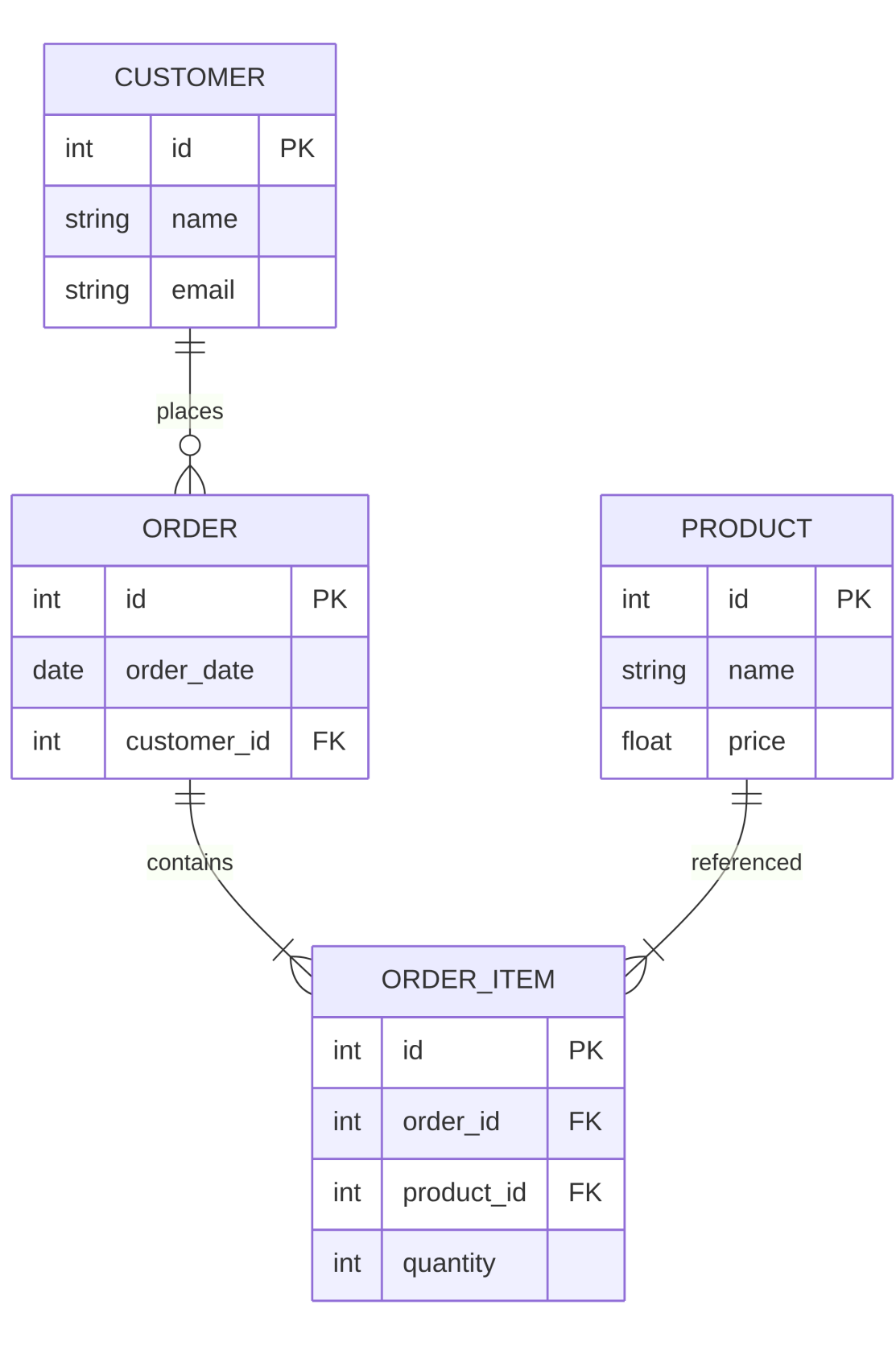
The CQRS Pattern

Motivation

In traditional application design, most systems interact with data using the **CRUD** model performing **C**reate, **R**ead, **U**pdate, and **D**elete operations. When designing these applications, we typically create **entity classes** and corresponding **repository classes** to handle database operations. In this approach, we use the **same model classes** for both read and write operations. Many applications work well with a straightforward CRUD model and don't require additional complexity.

Challenge: Differing Read and Write Requirements

Consider an application with the following **normalized tables**:

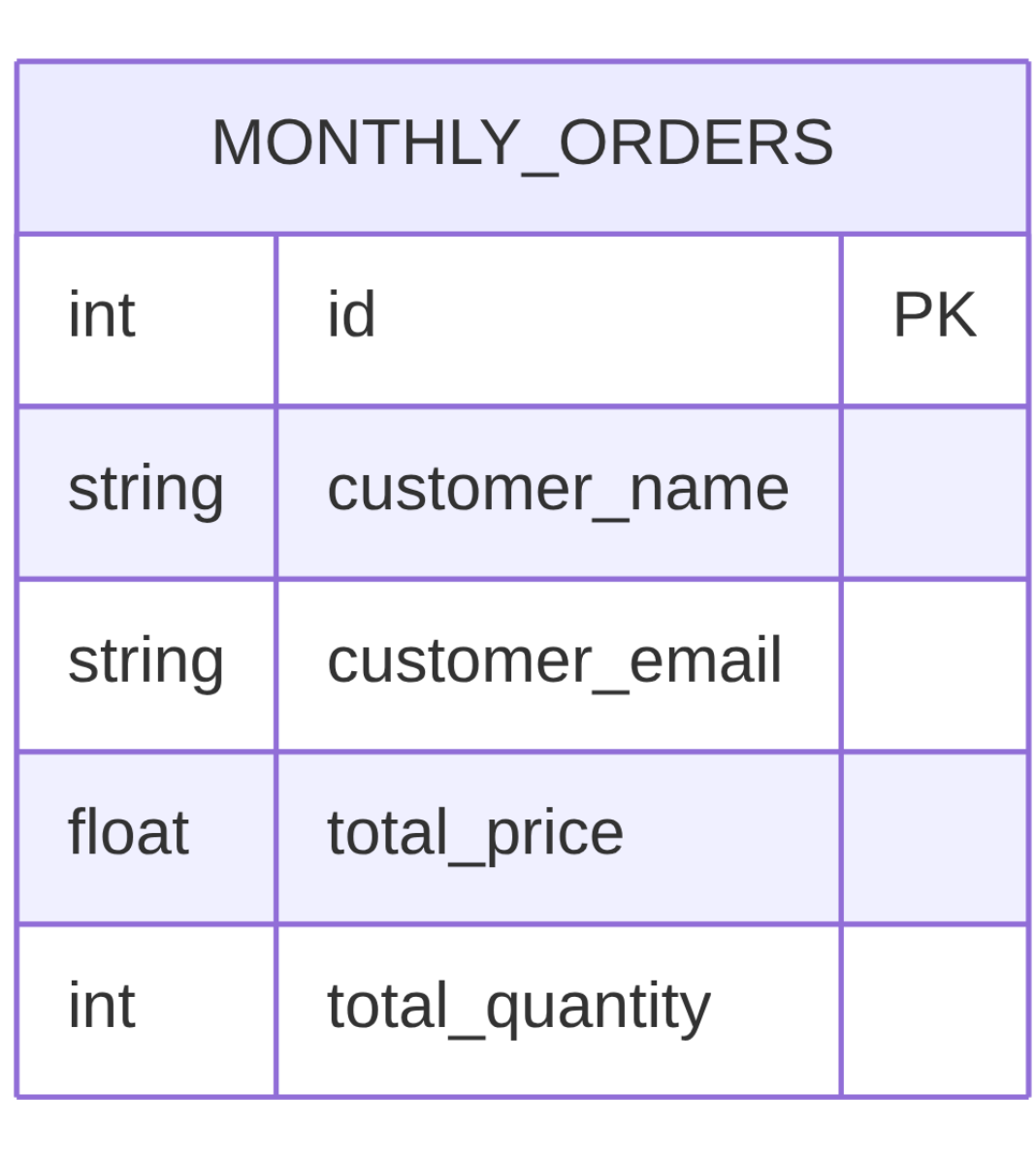

CUSTOMER
int id PK
string name
string email
places
ORDER
int id PK
date order_date
int customer_id FK
PRODUCT
int id PK
string name
float price
contains
referenced
ORDER_ITEM
int id PK
int order_id FK
int product_id FK
int quantity

Write operations, such as creating a new product, or order, are straightforward and involve inserting data directly into the relevant tables.

However, **read operations** often require more than just retrieving raw records (complex joins across multiple tables). For example:

- Fetching **all orders placed by a specific customer**.
- Calculating **state-wise sales**.
- Computing **product-wise sales**.

The more we normalize our data, the easier it becomes to write, but at the cost of making reads more complex and potentially slower.


MONTHLY_ORDERS
int id PK
string customer_name
string customer_email
float total_price
int total_quantity

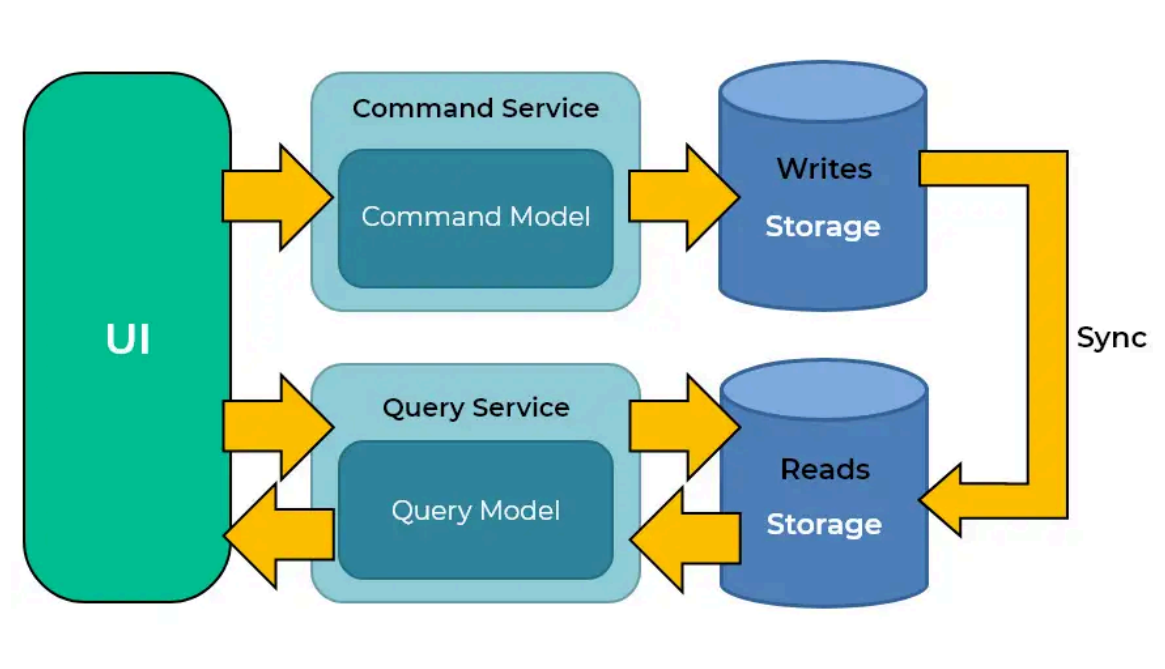
Challenge: Different traffic intensity between READ and WRITE operations

In most applications, **READ operations far outnumber WRITE operations**. This is especially true for web-based applications, which are typically **read-heavy**.

- **Social Media Platforms**: Users frequently check feeds, browse profiles, and search content — all of which are **read-heavy** operations.
- **Booking Applications**: Only a small percentage of users book tickets (writes), whereas the majority repeatedly search, check availability, and compare prices (reads). This creates a large disparity between **read and write workloads**.

CQRS (Command Query Responsibility Segregation)

The CQRS (Command Query Responsibility Segregation) pattern separates the responsibilities of handling commands (write operations) and queries (read operations) within a software system. By decoupling commands from queries, CQRS enables better performance tuning, fault isolation, and system responsiveness—making it a strong candidate for complex, high-throughput applications.


UI
Command Service
Command Model
Query Service
Query Model
Writes Storage
Reads Storage
Sync

To pattern suggests to **separate read and write concerns at the architectural level** and use dedicated microservices for **READ** and **WRITE** operations.

Benefits:

- **Independent Scaling**: Read and write services can be scaled separately based on demand. For example, in a flight-booking application, the read service can be scaled out to handle heavy traffic, while the write service remains relatively smaller.
- **Performance Optimization**: Each service can use different data storage strategies — denormalized views, caching, or read replicas for fast queries, while the write service ensures data integrity.
- **Fault Isolation**: A failure in the write service (e.g., a database insert issue) does not impact read operations, improving system availability.

Challenges:

- **Increased complexity:** implementing CQRS can introduce additional complexity to your system. You need to manage the flow of data between command and query models, potentially duplicating data for different models.
- **Eventual consistency:** CQRS can lead to eventual consistency issues, where query models may not reflect the most recent changes made by commands immediately. Dealing with this inconsistency requires careful handling and synchronization.
- **Data duplication:** CQRS often involves duplicating data between command and query models, leading to increased storage requirements and complexity in keeping data synchronized.

CQRS pattern and CAP theorem

CQRS is inherently neither AP nor CP, but its implementation can lean toward **AP** or **CP** depending on the consistency model used:

- **CQRS with CP**: Used in systems that require **immediate consistency**, ensuring that read and write operations are strictly synchronized (when sync after write is not available, reads fail).
- **CQRS with AP**: Common in **scalable, distributed systems** where reads are fast and based on an **eventually consistent** model (when sync after write is not available, reads might be inconsistent).

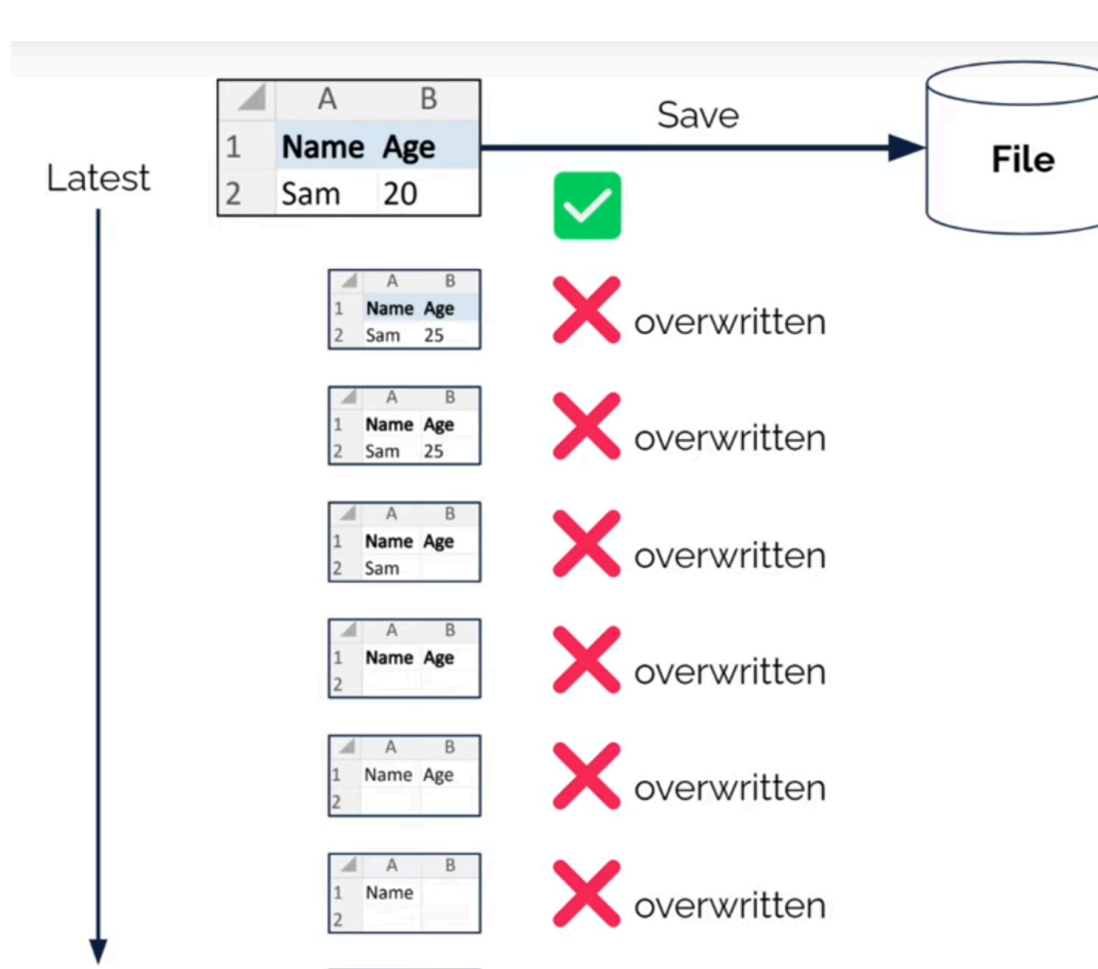
Event Sourcing

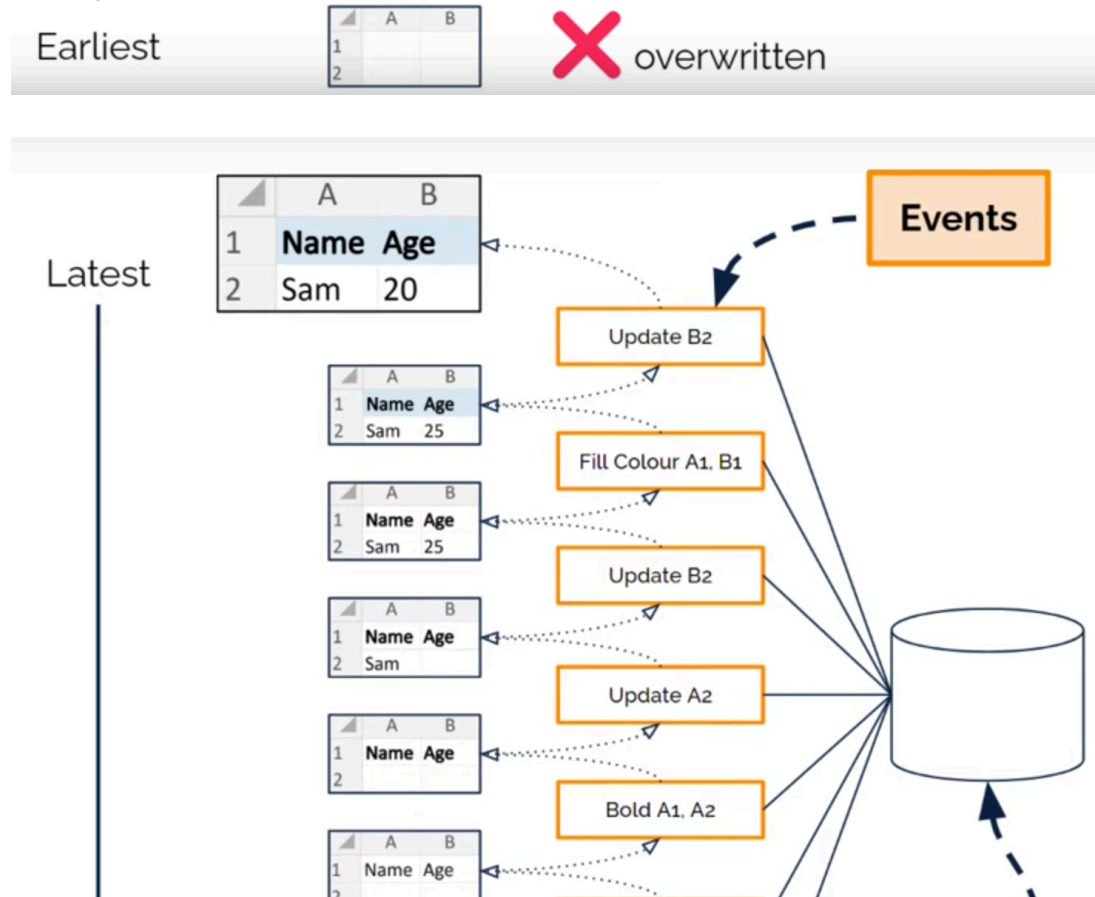
Motivation

In traditional CRUD-based systems, the state of an application is stored as the latest snapshot in a database. While simple, this approach has several limitations:

- **Loss of history**: Only the current state is stored; previous states are lost unless explicitly logged.
- **Complexity for auditing**: Reconstructing the history of changes requires additional logging or auditing mechanisms.
- **Challenges for distributed systems**: Maintaining consistency and traceability across services becomes harder in microservices.

Event Sourcing addresses these challenges by storing **all changes to the application state as a sequence of immutable events**, rather than storing just the current state. The current state can always be reconstructed by replaying the events.


Latest
Save
File
A B
1 Name Age
2 Sam 20
overwritten
overwritten
overwritten
overwritten
overwritten
overwritten
overwritten
Earliest
9


Latest
A B
1 Name Age
2 Sam 20
Events
Update B2
Fill Colour A1, B1
Update B2
Update A2
Bold A1, A2
Update B1
Update A1
Event Store
Earliest
11

Key Concepts

1. **Events**: Each event represents a state change in the system. For example, in an e-commerce system:
    - `OrderCreated`
    - `PaymentProcessed`
    - `InventoryUpdated`
    - `OrderShipped`
2. **Event Store**: A dedicated storage system that persists all events in order. It acts as the **source of truth** for the application state. Events are **append-only** and immutable.
3. **State Reconstruction**: The current state of a service or entity is reconstructed by replaying events from the event store. This allows for:
    - Auditing: Every change can be traced back to its originating event.
    - Temporal queries: Ability to view the system state at any point in time.
    - Debugging: Developers can replay events to reproduce past states or issues.
4. **Integration with CQRS**: Event Sourcing pairs naturally with CQRS:
    - **Write model (Command side)**: Emits events instead of directly updating the database.
    - **Read model (Query side)**: Builds projections (denormalized views) by consuming these events. These projections can be optimized for queries without affecting the write model.

Benefits

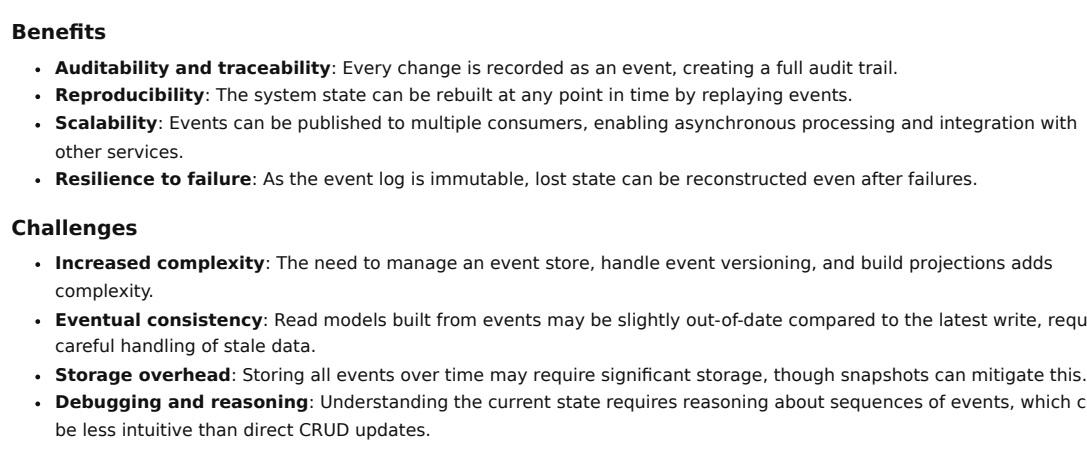
- **Auditability and traceability**: Every change is recorded as an event, creating a full audit trail.
- **Reproducibility**: The system state can be rebuilt at any point in time by replaying events.
- **Scalability**: Events can be published to multiple consumers, enabling asynchronous processing and integration with other services.
- **Resilience to failure**: As the event log is immutable, lost state can be reconstructed even after failures.

Challenges

- **Increased complexity**: The need to manage an event store, handle event versioning, and build projections adds complexity.
- **Eventual consistency**: Read models built from events may be slightly out-of-date compared to the latest write, requiring careful handling of stale data.
- **Storage overhead**: Storing all events over time may require significant storage, though snapshots can mitigate this.
- **Debugging and reasoning**: Understanding the current state requires reasoning about sequences of events, which can be less intuitive than direct CRUD updates.

Event Sourcing Example

For an e-commerce order service, an event-sourced workflow might look like this:


CustomerOrderService
PaymentService
InventoryService
ShippingService
OrderCreated
PaymentProcessed
InventoryReserved
OrderShipped
EventStore
UpdateProjections
NotifyOtherServices
OrderReadModel
NotificationService

- **Command side**: Each service emits events to the **Event Store**.
- **Query side**: Events are consumed to update **read models** or trigger notifications.

Event Sourcing and CAP Theorem

Event Sourcing can be implemented in both **AP** and **CP** systems, depending on the consistency guarantees of the event store and read models:

- **AP systems**: Read projections are eventually consistent with the write model; services continue to operate during partitions.
- **CP systems**: Writes may block during network partitions to ensure the event log remains strictly consistent.

References

- https://www.orkes.io/what-is-conductor
- https://www.baeldung.com/cqrs-event-sourcing-java
- https://www.baeldung.com/orkes-conductor-saga-pattern-spring-boot
- https://www.baeldung.com/java-domain-driven-design-event-sourcing
- https://martinfowler.com/eaaDev/EventSourcing.html